

Project Report

on

First Park Then Pay

By

Udhayarajan J

Under the Guidance of

Nidhi Damle

Master of Computer Application

P.E. Society's

Modern Institute of Business Studies (Autonomous)

Pune

2025-2026



Progressive Education Society's
Modern Institute of Business Studies (Autonomous)

Certificate

This is to certify that the Mini Project (MMPRJ-2511) entitled **First Park Then Pay** is submitted by **UDHAYARAJAN J** a Bonafede student of this institute, studying for a Master of Computer Application, has successfully carried out MCA semester-I Mini Project. This project report is submitted in partial fulfilment of **M.C.A. Semester I Academic Year 2025-26** curriculum as per Institute norms.

Dr. Maithili Arjunwadkar
Director
PES MIBS

Project Guide

Internal Examiner

ACKNOWLEDGEMENT

It is my proud privilege to express my profound gratitude to the entire management of Progressive Education Society's Modern Institute of Business Studies (Autonomous) and Faculty members of the institute for providing me with the opportunity to avail the excellent facilities and infrastructure of the institute. The knowledge and values inculcated have proved to be of immense help to my career.

I am grateful to **Prof. Dr. Maithili Arjunwadkar (Director, MIBS)** for their astute guidance, constant encouragement and sincere support for this project work.

I express my sincere gratitude to **Nidhi Damle (Assistant Professor, MCA Department, MIBS)** for their inspiration, constructive suggestion, mastermind analysis and affectionate guidance in my work, without which this project work completion would have been impossible for me. Sincere thanks to all my seniors and colleagues at the company for their support and assistance throughout the project.

I would like to thank all the faculties for providing me with an opportunity to pursue my industrial training, as it is an important part of the MCA course.

I feel proud and privileged in expressing my deep sense of gratitude to all those who have helped me in presenting this assignment. I would be failing in my endeavour if I do not place my acknowledgement.

INDEX

Chapter	Details	Page No
1	Introduction	
	1.1 Problem Statement	1
	1.2 Objectives of the Project	1
	1.3 Scope of the Project	1
	1.4 Technologies Used	2
	1.5 System Overview	2
2	Literature Review / Existing System	
	2.1 Existing System	3
	2.2 Limitations of Existing System	3
	2.3 Proposed System	3
3	System Analysis	
	3.1 Functional Requirements	4
	3.2 Non-Functional Requirements	4
	3.3 Feasibility Study	4
	3.4 Use Case Diagrams	5
	3.5 System Architecture	5
4	System Design	
	4.1 ER Diagram / Database Design	6
	4.2 User Interface Design (Screenshots / Prototypes)	7
5	Implementation	
	5.1 Coding Environment	10
	5.2 Module Description	11
6	Conclusion & Future Enhancements	
	6.1 Conclusion	12
	6.2 Future Scope / Enhancements	12
7	Bibliography	
	7.1 References	13

Chapter 1 – Introduction

1.1 Problem Statement

Urban areas face increasing challenges in managing parking spaces due to rapid vehicle growth. Traditional parking systems rely heavily on manual processes, leading to issues such as inaccurate slot allocation, congestion at entry/exit points, lack of transparency in billing, and inefficient tracking of available spaces. Users often struggle to find free parking slots, and administrators find it difficult to monitor usage in real-time. A digital, automated, and secure solution is required to streamline the parking workflow.

1.2 Objectives of the Project

- Provide real-time parking slot availability to all users.
- Allow customers to park first and pay later, ensuring a transparent billing process.
- Implement secure user authentication and authorization using Keycloak.
- Maintain centralized data for users, parking slots, payments, and history.
- Build a scalable backend using Spring Boot REST APIs.
- Deploy the application using Docker for consistency and portability.
- Provide admin dashboards for monitoring vehicle count, payments, and slot usage.

1.3 Scope of the Project

The project is designed as a complete digital parking management system for small, medium, and large parking providers. It covers:

- Secure user login and role-based access
- Real-time slot management (available, booked, occupied)
- Vehicle type and slot configuration
- Billing based on parking duration
- Payment tracking (paid/pending)
- Admin dashboards and reports

Out of Scope / Future Scope:

- Integration with IoT sensors for automatic slot detection
- Mobile app for end users
- Online payment gateway integration (UPI, cards)
- AI-based parking prediction

1.4 Technologies Used

- **Frontend:** HTML5, CSS3, JavaScript / Angular
- **Backend:** Java 21, Spring Boot
- **Authentication:** Keycloak
- **Database:** MySQL 8.0
- **DevOps:** Docker, Git, GitHub
- **Testing Tools:** Postman
- **Build Tool:** Gradle

1.5 System Overview

The system enables users to view parking slot availability, select vehicle type, park vehicles, and pay after leaving. Administrators can configure vehicle types, blocks, and slots and manage payments. The backend is built using REST APIs, with secure token-based authentication via Keycloak. All services run in Docker containers.

Chapter 2 – Literature Review / Existing System

2.1 Existing System

Traditional parking systems involve:

- Manual entry and exit recording
- Paper-based receipts
- No real-time availability tracking
- Manual billing calculations
- High chance of errors

2.2 Limitations of Existing System

- Time-consuming and prone to human error
- No transparency in billing
- Congestion due to manual processes
- Limited monitoring capability
- Poor scalability

2.3 Proposed System

The “First Park Then Pay” system modernizes parking management by providing:

- Digital slot tracking
- Automated billing
- Secure user login
- Dashboard-based monitoring
- Real-time status updates

Chapter 3 – System Analysis

3.1 Functional Requirements

User Requirements:

- Login & authentication
- View available parking slots
- Select vehicle type
- Park vehicle
- Pay after parking duration
- View history

Admin Requirements:

- Manage vehicle types
- Create blocks and slot numbers
- View parking activity
- Manage payments
- Access dashboards

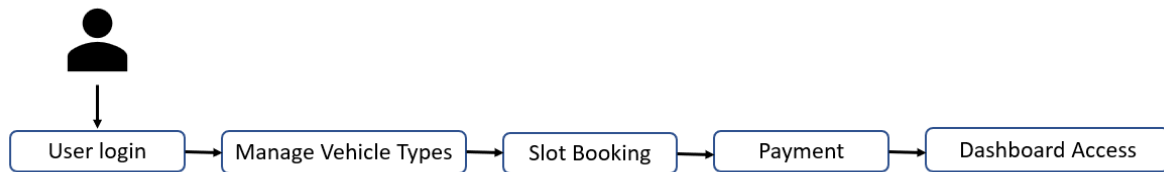
3.2 Non-Functional Requirements

- **Security:** Token-based authentication, HTTPS-ready
- **Performance:** Faster loading and real-time updates
- **Usability:** Simple UI and intuitive navigation
- **Reliability:** Consistent system behaviour
- **Scalability:** Support increasing users and slots

3.3 Feasibility Study

- **Economic Feasibility:** Requires only basic infrastructure
- **Technical Feasibility:** Uses widely adopted technologies like Spring Boot, MySQL
- **Operational Feasibility:** Easy to operate with dashboards

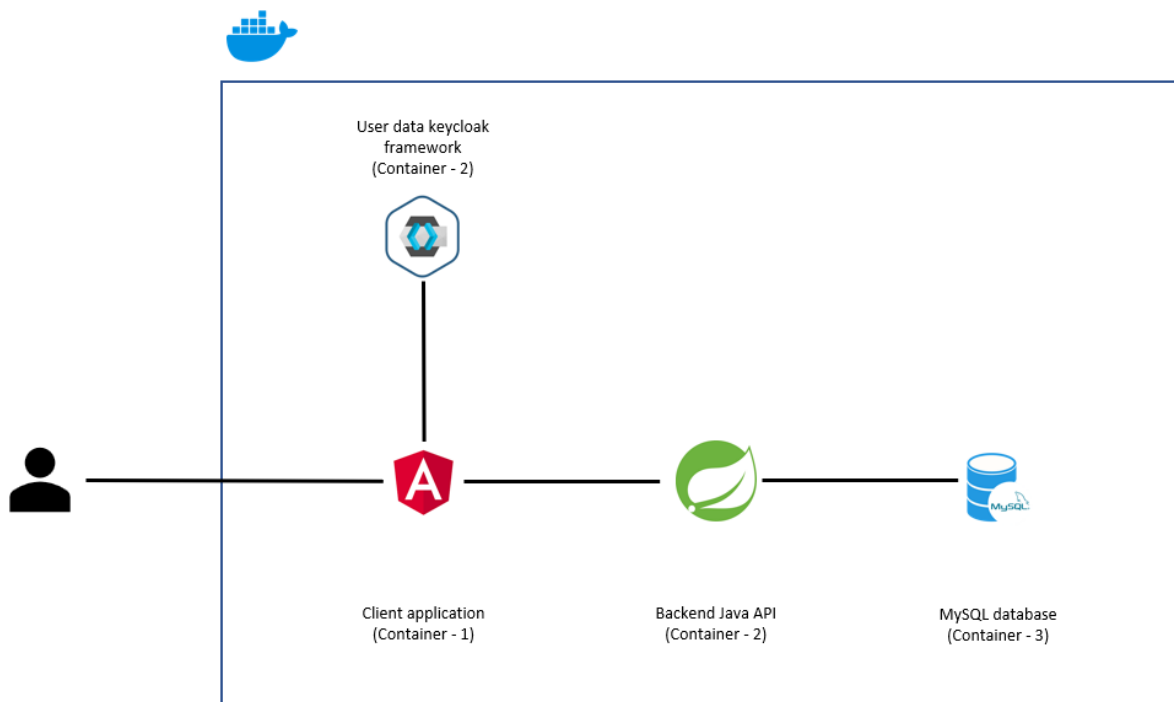
3.4 Use Case Diagram



3.5 System Architecture

A typical three-tier architecture:

- **Presentation Layer:** Browser / UI
- **Application Layer:** Spring Boot APIs
- **Data Layer:** MySQL
- **Security Layer:** Keycloak
- **Deployment:** Docker containers

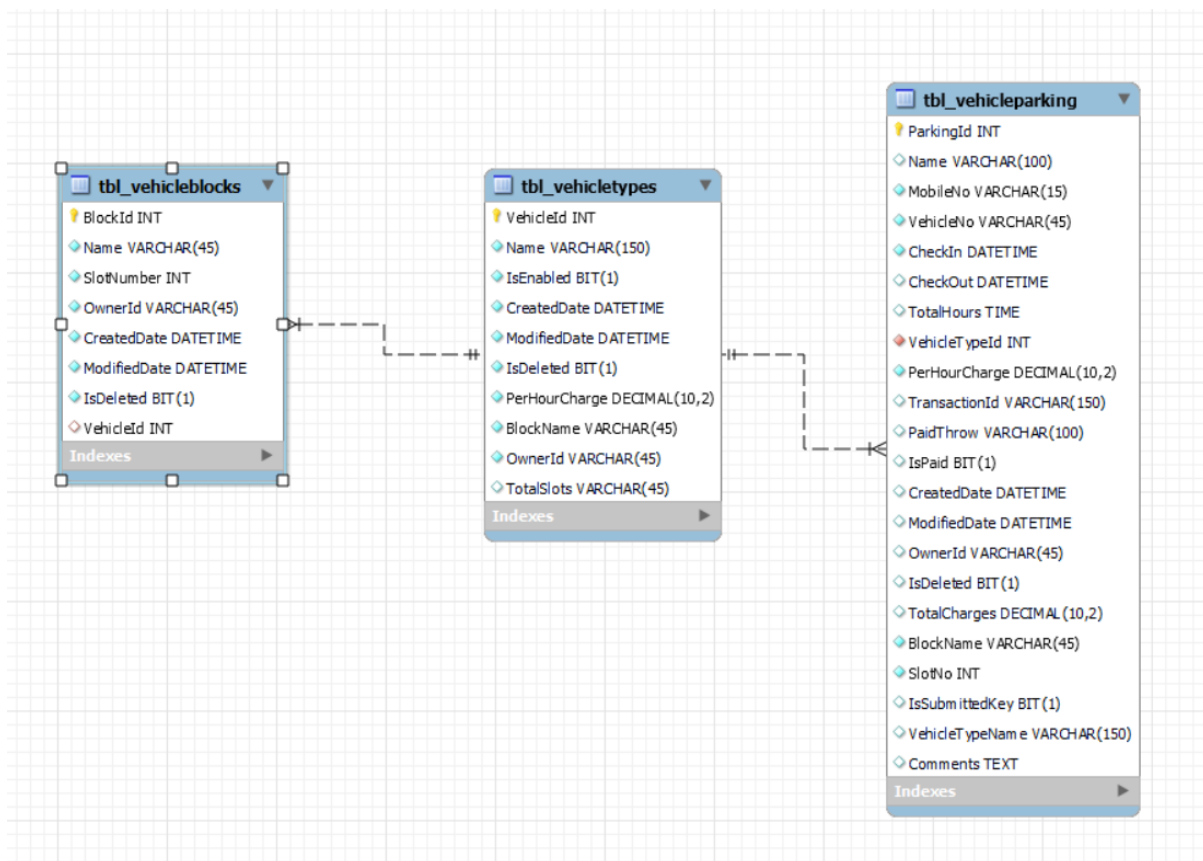


Chapter 4 – System Design

4.1 ER Diagram / Database Design

Main entities:

- Vehicle Types
- Parking Slots
- Billing & Payments



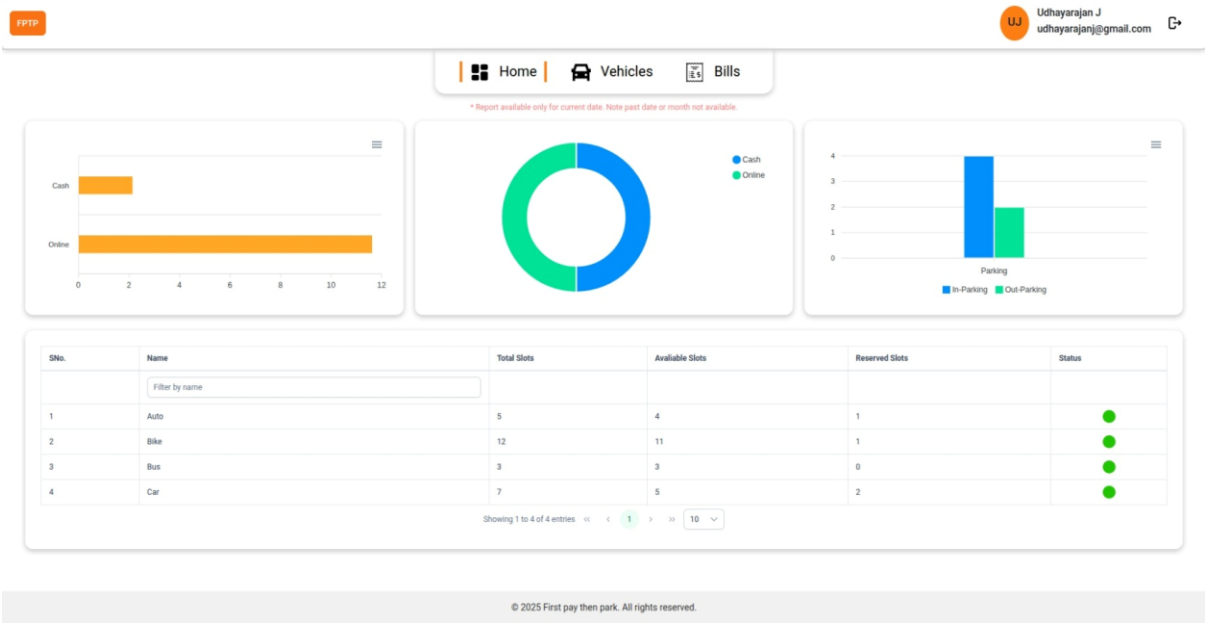
4.2 User Interface Design

Using key cloak to manage personal details and user

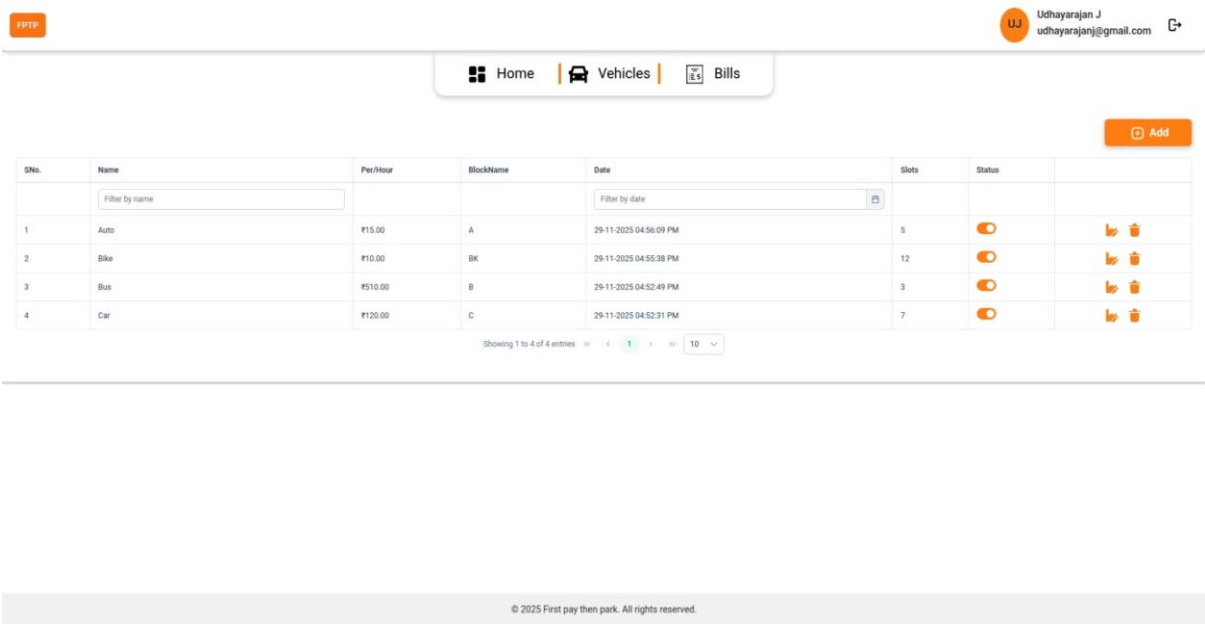
The screenshot shows the Keycloak user interface. On the left is a dark sidebar with a menu containing 'Personal info', 'Account security', 'Applications', and 'Organizations'. The 'Personal info' section is active. The main content area is titled 'Personal info' with the subtitle 'Manage your basic information'. It features a 'General' tab, which is selected, and a 'Jump to section' dropdown menu. The form contains the following fields: 'Username' (value: udhayarajan@gmail.com), 'Email' (value: udhayarajan@gmail.com), 'First name' (value: Udhayarajan), 'Last name' (value: J), and 'UPI ID' (value: udhayarajan080gokhdfcbank). At the bottom of the form are 'Save' and 'Cancel' buttons.

This is an identical screenshot to the one above, showing the Keycloak 'Personal Info' page with the 'General' tab selected. The form fields and their values are: Username (udhayarajan@gmail.com), Email (udhayarajan@gmail.com), First name (Udhayarajan), Last name (J), and UPI ID (udhayarajan080gokhdfcbank). The 'Save' and 'Cancel' buttons are at the bottom.

Dashboard & Reporting view



Manage vehicle types



Manage billing and payments view

FFIP

UJ

Udhayarajan J
udhayarajanj@gmail.com

Home

Vehicles

Bills

Vehicle No.

Enter vehicle number

Owner Name (optional)

Enter owner name

Mobile Number (optional)

Enter mobile number

Vehicle type

- Select -

Check in

-

Block Name

Select vehicle type then Check whether available slots are not ?

Slot No.

Select vehicle type then Check whether available slots are not ?

Per/Hour Charge.

Select vehicle type.

Is key submitted ?

SNo.

Vehicle No.

Name

Mobile No.

Check In

Check Out

Total Hours

Vehicle Type.

Block Name

Slot No.

Paid

Payment Mode

Key Status

Filter by vehicle number

Filter by name

Filter by mobile number

Filter by check in date

Filter by check out date

- Select -

- Select -

- Select -

- Select -

1	MH15K999	-	-	29-11-2025 05:00:07 PM		00:00:17	Car	C	1		-		
2	TN78UJ7554	-	-	29-11-2025 04:59:35 PM		00:00:45	Auto	A	1		-		
3	TN58OL9887	-	-	29-11-2025 04:59:04 PM	29-11-2025 05:00:29 PM	00:01:22	Bus	B	1		Online		
4	TN67YH8989	-	-	29-11-2025 04:58:50 PM		00:00:35	Car	C	2		-		
5	MH15K999	-	-	29-11-2025 04:58:38 PM	29-11-2025 04:59:45 PM	00:01:04	Car	C	1		Cash		
6	MH14LK0511	Udhayarajan J	8248748016	29-11-2025 04:56:52 PM		00:00:00	Bike	BK	1		-		

Showing 1 to 6 of 6 entries

© 2025 First pay then park. All rights reserved.

9

Chapter 5 – Implementation

5.1 Coding Environment & Algorithms

The implementation of the “First Park Then Pay” system is built using a modern technology stack that ensures scalability, reliability, and maintainability.

Backend Environment – Java 21 with Spring Boot

- The backend is developed using **Java 21**, the latest LTS version, offering improved performance and modern language features.
- **Spring Boot** is used to build RESTful APIs that handle all business logic, database operations, authentication, and communication with the frontend.
- Key features of the backend:
 - REST API development
 - Dependency injection
 - Exception handling
 - Database connectivity using Spring Data JPA
 - Service-oriented architecture

Frontend Environment – Angular

Angular is used as the frontend framework to build the web application's user interface. It communicates with the Spring Boot backend using REST APIs.

How Angular Works in the Project:

- Angular provides a dynamic, component-based UI.
- It consumes backend APIs using **HTTP Client**.
- Forms are used for vehicle type creation, slot booking, and payment entry.
- Angular services store temporary data like authentication tokens.

Interaction Between Angular & Java (Spring Boot):

1. User interacts with Angular UI.
2. Angular sends HTTP requests (GET/POST/PUT/DELETE) to Spring Boot APIs.
3. Spring Boot processes the request and interacts with MySQL.
4. The backend returns JSON responses.
5. Angular updates the UI based on the response.

JWT-Based Authentication Using Keycloak

- Users authenticate via Keycloak login.
- Keycloak issues a **JWT token** which Angular stores in local/session storage.
- Angular attaches the JWT token in the header for each API request: Authorization: Bearer <token>
- Spring Boot validates the token using Keycloak adapters.

Parking Charge Calculation Algorithm

The system calculates the total parking bill based on duration and rate per hour.

Formula: $\text{totalCharge} = \text{hoursParked} \times \text{ratePerHour}$

Steps:

1. Record entry time.
2. Record exit time.
3. Calculate total time spent.
4. Fetch vehicle type's hourly rate.
5. Multiply duration $\times$ rate.
6. Generate bill and update payment status.

5.2 Module Description

1. User Management Module

- Managed via Keycloak
- Role-based access: Admin, Customer

2. Vehicle Management Module

- Create/manage vehicle types
- Set hourly charges
- Configure blocks and slots

3. Billing and Payment Module

- Auto calculates duration-based charges
- Supports Cash/Online entry
- Validates paid status

4. Dashboard Module

- Real-time parking data
- Payment summary
- Slot usage statistics

Chapter 6 – Conclusion & Future Enhancements

6.1 Conclusion

The “First Park Then Pay” system offers a modern, reliable, and efficient solution for smart parking management. It removes manual dependencies, enhances billing transparency, and provides powerful dashboards for administrators. With Spring Boot, Keycloak, and Docker, the system is scalable, secure, and future-ready.

6.2 Future Enhancements

- IoT-based slot automation
- Mobile app development
- Online payment gateway integration
- AI-based parking prediction

Chapter 7 – Bibliography

7.1 References

- **Angular Framework Documentation**
<https://angular.io/docs>
- **Node.js Official Documentation**
<https://nodejs.org/en/docs>
- **Java 21 Documentation (Oracle)**
<https://docs.oracle.com/en/java/>
- **Spring Boot Framework Documentation**
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- **Keycloak Documentation**
<https://www.keycloak.org/documentation>
- **MySQL Database Documentation**
<https://dev.mysql.com/doc/>
- **Docker Documentation**
<https://docs.docker.com/>
- **Visual Studio Code Documentation**
<https://code.visualstudio.com/docs>
- **IntelliJ IDEA Community Edition Documentation**
<https://www.jetbrains.com/idea/documentation/>
- **Project Source Code (GitHub Repository)**
<https://github.com/UdhayarajanJ/2202561-MCA-2025-2027.git>